

Overview

The TCP server is designed to handle two primary categories of tasks: CPU-intensive and I/O-intensive. Each incoming client connection is processed concurrently, with tasks dynamically categorized based on the nature of the request. This differentiation enables the system to efficiently execute CPU-bound and I/O-bound operations in parallel, improving overall responsiveness and efficiency of the system.

Concurrency Paradigms

Proposed Level of Concurrency: Task-Level

We implemented multi-threading and asynchronous I/O using the Tokio runtime. The core concurrency paradigm involves:

- Async-await is used to efficiently manage lightweight I/O-bound operations without blocking threads. These include tasks such as accepting TCP connections, reading from sockets, and I/O intensive tasks.
- Blocking threads (via `tokio::task::spawn_blocking`) for CPU-intensive computations, preventing the Tokio runtime from stalling due to thread-blocking operations.

Each client connection is handled on a per-connection async task spawned by `tokio::spawn`. Hence, multiple client connections can be accepted and processed concurrently, minimally achieving *Client-Level Concurrency*. Inside each async task (client connection), the client's input is read asynchronously, and depending on the task type, is either awaited as an async function or delegated to a separate blocking thread.

CPU-intensive tasks are moved off the async runtime using a dedicated blocking thread pool to preserve responsiveness for I/O tasks, while I/O intensive tasks are executed asynchronously, on the asynchronous thread pool. Hence these 2 task types can be executed concurrently. This design allows concurrent handling of multiple TCP client connections and a mix of I/O-bound and CPU-bound operations across connections, achieving *Task-Level Concurrency*.

The effective concurrency level may decrease in the following cases:

- Blocking thread pool saturation: If more than the specified limit of 12 threads (in this case, the maximum set for the blocking thread pool) are executing concurrently, any additional CPU-intensive tasks will be queued. These tasks will remain in the queue until threads become available for execution.
- Limited CPU cores: Even with sufficient threads, if the system has limited physical cores, true parallel execution of CPU tasks will be restricted
- I/O bottlenecks: If client connections are slow or unresponsive, they may hold up their individual tasks, though they won't block others due to async I/O

Previous Implementations

In an earlier implementation, all tasks including CPU-intensive ones were handled by the Tokio asynchronous threadpool, which was limited to 16 worker threads. While this setup performed adequately with a small number of clients and messages, the performance began to degrade as the load increased. When `spawn_blocking` was introduced to offload CPU-bound tasks onto dedicated blocking threads, we observed a marked improvement in performance under high load.